

Software Re-Engineering

Week 7

Quality in Software Re-Engineering

Quality



- In software, quality generally refers to how well the software meets the needs and expectations of users and stakeholders with a focus on key characteristics like:

Key Quality Attributes in SRE

Reliability

Usability

Performance

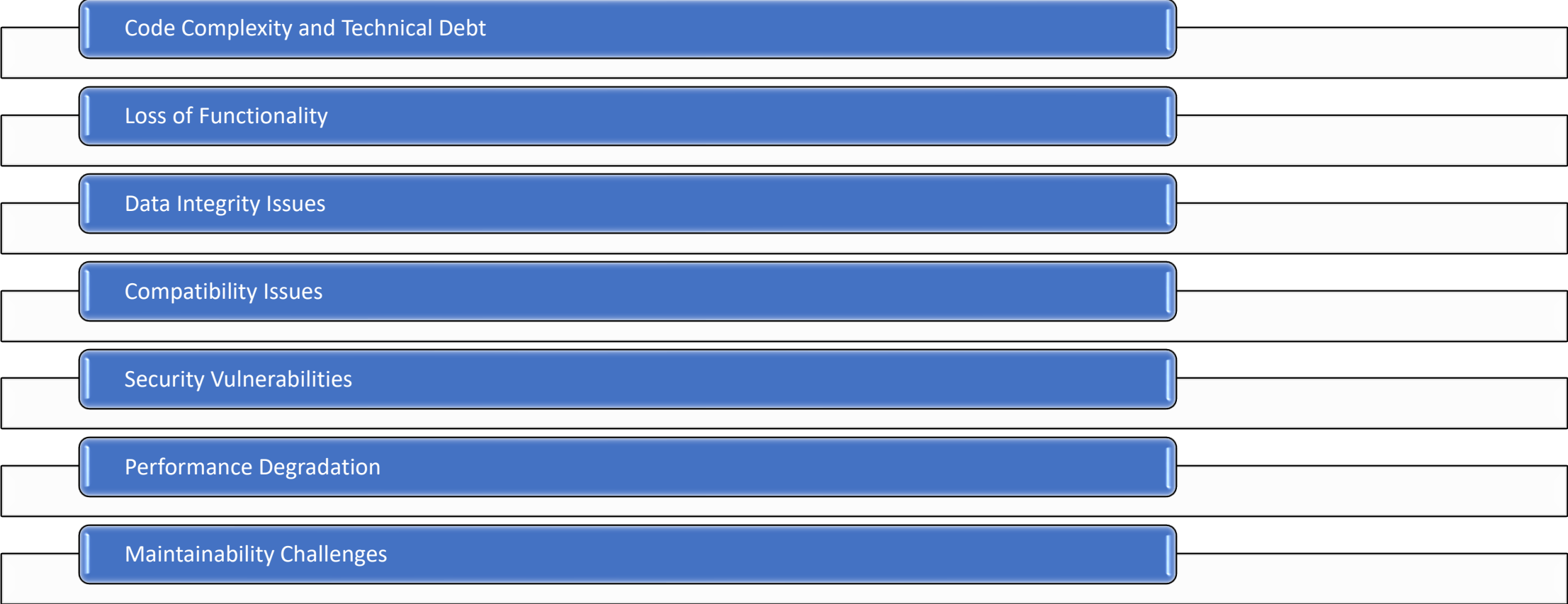
Maintainability

Scalability

Key Quality Attributes in SRE

- **Reliability:** Consistency in performance and absence of critical failures.
 - Importance: Re-engineered systems need to be as dependable as the legacy systems they replace.
- **Usability:** Ease with which users can interact with the software.
 - Importance: Even if functionality is retained, poor usability can lead to lower adoption rates and operational inefficiency.
- **Performance:** Speed and responsiveness of the software.
 - Importance: Performance should either match or exceed that of the legacy system to avoid user frustration.
- **Maintainability:** Ease of making updates or fixing bugs.
 - Importance: A re-engineered system should be maintainable with minimal effort, enabling future enhancements.
- **Scalability:** The ability to handle growth in users, data, and processes.
 - Importance: Scalability is crucial for supporting future business needs.

Common Quality Challenges in SRE



Code Complexity and Technical Debt

Loss of Functionality

Data Integrity Issues

Compatibility Issues

Security Vulnerabilities

Performance Degradation

Maintainability Challenges

Code Complexity and Technical Debt

- Legacy systems often contain large amounts of unoptimized, poorly structured code accumulated over years.
- Frequent updates, bug fixes, and feature additions without refactoring lead to complex, twisted code that is difficult to modify or understand.
- This complexity increases the risk of errors during re-engineering

Code Complexity and Technical Debt

- Consider a telecommunications company with a legacy billing system written in a mix of Java and SQL. Over years, different development teams added numerous patches, custom workflows, and calculations for new services. As a result:
 - The system contains duplicate code snippets, with similar billing calculations written in slightly different ways across various modules.
 - Dependencies are deeply nested, where a change in one part of the code (e.g., the billing rates) can cause unexpected issues in other sections like discounts or tax calculations.
 - Many functions are not modular; instead, they are monolithic blocks of code performing multiple tasks, making it difficult to isolate and test specific functionalities.

Code Complexity and Technical Debt

- Technical Challenge:
 - Refactoring to reduce code duplication and complexity without disrupting core functions.
 - De-coupling dependencies to create modular components for easier testing and updates.
- Solution Strategy:
 - Use a static analysis tool like SonarQube, Lint, or aDoctor to identify code smells, duplication, and dependencies. Refactor duplicate calculations into shared functions, modularize large monolithic functions, and break complex dependencies incrementally to improve readability and maintainability.

Loss of Functionality

- When re-engineering a system, critical features or workflows might unintentionally be removed or altered, especially if they are undocumented. These features may include "hidden" functionalities that users have relied on but aren't explicitly defined in the system documentation.
 - A healthcare application originally built for patient records also includes an undocumented function that auto-calculates specific dosage recommendations for prescriptions. During re-engineering, the development team focuses on documented functions only, unaware of this feature. When the new system goes live, users complain that dosage recommendations are missing, causing issues in patient care.

Loss of Functionality

- Technical Challenge:
 - Detecting and documenting all functionalities, including hidden ones. Ensuring that test cases cover these hidden features before re-engineering begins.
- Solution Strategy:
 - Conduct thorough interviews with end-users and stakeholders to identify and document all functions. Use test-driven development to create tests cases that ensure functionalities are retained.

Data Integrity Issues

- Data in legacy systems often have a different structure, schema, or format from modern systems. During re-engineering, migrating this data without losing or corrupting it is challenging. Data integrity issues can arise if mappings between old and new formats aren't handled correctly.
 - A retail company re-engineers its customer relationship management system. In the legacy system, addresses are stored as a single text field, while the new system requires separate fields for street, city, state, and zip code. During migration, errors occur, such as:
 - Misplaced data where zip codes are stored in the city field.
 - Data loss if incomplete addresses don't map correctly to the new fields.

Data Integrity Issues

- Technical Challenge:
 - Accurately mapping old data structures to new ones without loss or corruption.
 - Validating data integrity at each stage of the migration.
- Solution Strategy:
 - Use data migration tools like Talend or Informatica, which can help manage and validate data transformations, ensuring consistency between the old and new databases. Implement a comprehensive data validation process to check data accuracy post-migration.

Compatibility Issues

- Legacy systems may be deeply integrated with outdated hardware, software, or APIs that aren't compatible with modern technology. Re-engineering can break these integrations, causing incompatibility with other dependent systems.
 - Consider an insurance company using a legacy claims processing system that integrates with an old mainframe-based document management system. The re-engineered application is moved to a cloud environment, which doesn't support the mainframe's custom file transfer protocol. As a result,
 - The new system cannot send or receive documents from the old mainframe system.
 - Claims processing workflows are disrupted, impacting operations.

Compatibility Issues

- Technical Challenge:
 - Ensuring compatibility between the new system and existing external systems.
 - Maintaining integrations without extensive rework of dependent systems.
- Solution Strategy:
 - Use an adapter pattern to create “wrapper” modules that convert legacy communications into formats the new system can interpret. APIs like MuleSoft or Apache Camel can facilitate integration between different systems.

Security Vulnerabilities

- Legacy systems often lack modern security practices, such as encryption, proper authentication, and authorization. During re-engineering, any new integrations, ports, or permissions added without careful security considerations can create vulnerabilities.
 - A manufacturing company re-engineers its supply chain system, allowing it to communicate with external vendors via a web API. However, the legacy system was never designed with network security in mind. It lacks SSL/TLS encryption, and data is sent in plain text. The result:
 - Sensitive information, such as vendor pricing and inventory levels, becomes vulnerable to interception.
 - Lack of secure authentication could allow unauthorized access to the system.

Security Vulnerabilities

- Technical Challenge:
 - Identifying and patching legacy security flaws.
 - Ensuring new communication channels are secure and don't expose sensitive data.
- Solution Strategy:
 - Conduct a security audit before re-engineering. Use secure APIs, encryption protocols like TLS, and ensure that data is encrypted both in transit and at rest. Modernize authentication methods using token-based protocols like OAuth 2.0.

Performance Degradation

- Re-engineering can lead to performance issues if the new architecture or technology stack isn't optimized for the application's needs. Modern systems may have more layers (e.g., microservices), which could introduce latency if not properly managed.
 - A financial firm re-engineers its trading platform, moving from a monolithic architecture to a microservices-based one. However:
 - Each service call adds network latency, which didn't exist in the monolithic version.
 - The end-to-end transaction time increases, leading to slower processing of trading data and order fulfillment.

Performance Degradation

- Technical Challenge:
 - Optimizing new architectures to handle expected loads without introducing excessive latency.
 - Balancing modularity with performance, especially for real-time systems.
- Solution Strategy:
 - Conduct load testing using tools like Apache JMeter or Locust to simulate high loads and measure system response times. Employ performance optimization techniques such as caching, asynchronous processing, and minimizing the number of API calls.

Maintainability Challenges

- In legacy systems, maintenance is challenging due to outdated programming languages, lack of modularization, and high coupling between modules. Re-engineering to improve maintainability requires careful planning to avoid reintroducing these issues.
 - A transportation company re-engineers its scheduling software, initially built in an obsolete language like COBOL, to a modern language like Java. However:
 - Without proper modularization, the re-engineered system retains tight coupling, where changes in one module (e.g., the route calculation) impact unrelated modules.
 - This makes future maintenance nearly as difficult as the original system, failing the objective of improving maintainability.

Maintainability Challenges

- Technical Challenge:
 - Ensuring re-engineered code is modular and loosely coupled.
 - Using design patterns and architecture principles to improve maintainability.
- Solution Strategy:
 - Use design principles like SOLID and architectural patterns like microservices or layered architecture. This ensures each module can be updated independently, reducing future maintenance complexity.

SOLID Principles

- **S**ingle Responsibility Principle: A class should have only one reason to change, meaning it should have only one responsibility.
- **O**pen/Closed Principle: Software entities (classes, modules, functions) should be open for extension but closed for modification.
- **L**iskov Substitution Principle: Objects of a superclass should be replaceable with objects of a subclass without altering the correctness of the program.
- **I**nterface Segregation Principle: A client should not be forced to implement interfaces it does not use.
- **D**ependency Inversion Principle: High-level modules should not depend on low-level modules.



Questions?